

Churn Prediction Model Performance & Reliability

What the model finds, how far it can be trusted, and how to put it to work

XGBoost classifier | Target: 90-day churn | 64,738 subscribers | Prepared September 2026

HEADLINE RESULTS (held-out test set, 9,711 subscribers)



In one paragraph

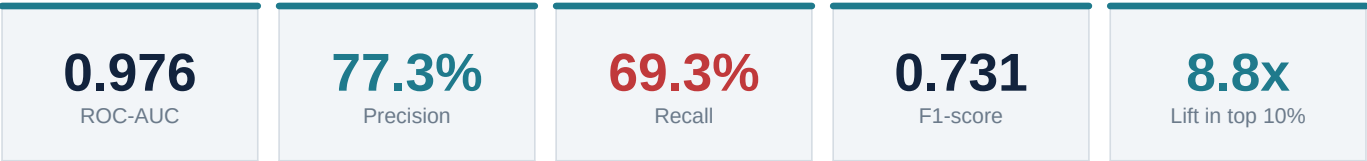
We trained a machine-learning model (XGBoost) to predict which subscribers will leave within the next 90 days. It was built on 64,738 subscribers, tuned on a separate validation set, and then scored once on a further 9,711 subscribers it had never seen. It ranks customers by churn risk very well, its accuracy holds up from training to unseen data (no meaningful overfitting), and it is not under-powered (no underfitting). The result is also free of "too good to be true" leakage, which we tested explicitly.

Inside this report

1	Executive summary	What was built, what it delivers, model health scorecard
2	Data & approach	Dataset, 70/15/15 split, leakage control, handling the 1:19 class imbalance
3	Model performance	Accuracy, precision, recall, F1, ROC-AUC in plain English
4	Overfitting & underfitting	Nine independent checks that the model is neither memorising nor under-learning
5	Business impact	Risk tiers, campaign sizing, revenue at stake, break-even economics
6	Churn drivers & dependency	What the model pays attention to, and the one signal to validate
7	Risks, recommendations, next steps	Honest limitations and a practical rollout plan
A	Appendix	Method, hyper-parameters, glossary

1. Executive summary

What the model delivers, in the language of the business



What this means for the business

- **Focus the effort.** Contacting only the riskiest 10% of subscribers reaches **88%** of everyone who goes on to churn - 8.8 times better than picking customers at random.
- **Flags you can act on.** At the recommended operating point the model flags 436 of 9,711 customers (4.5%). **77 in every 100 flagged customers are genuine churners**, and the model finds **69 in every 100** real churners.
- **Very few false alarms.** Only 99 of 9,225 loyal customers were wrongly flagged (1.1%), so retention budget is rarely wasted on people who would have stayed.
- **Scores mean something.** Customers scored 60%+ churned 86% of the time; customers scored under 5% churned 0.5% of the time. The score can be used directly to tier the customer base.
- **Trustworthy, not artificial.** All three outcome-linked columns were excluded, the model was scored on unseen data, and a label-shuffling test collapsed to coin-flip level (ROC-AUC 0.52), which is strong evidence that there is no hidden leakage.

Model health scorecard

Check	Result	Status
Ranking power (ROC-AUC on unseen data)	0.976 (95% CI 0.971 - 0.981)	PASS
Overfitting: train vs test ROC-AUC gap	0.001 (concern starts above ~0.05)	PASS
Underfitting: train performance vs baselines	Train PR-AUC 0.847 vs 0.050 random; no gain from deeper models	PASS
Stability across 30 tuning trials	Validation ROC-AUC 0.968 - 0.972	PASS
Leakage test (shuffled labels)	ROC-AUC 0.516 (expected 0.50)	PASS
Calibration (avg. predicted vs actual churn)	5.2% predicted vs 5.0% actual	PASS
Dependence on one signal (MNP enquiry flag)	F1 falls 0.73 to 0.58 without it; confirm timing	WATCH

Bottom line

The model is a **good fit**: it is neither memorising the training data (overfitting) nor missing patterns (underfitting). The single item to validate with the business before go-live is *when* the number-porting (MNP) enquiry flag is recorded relative to the actual port-out, because it is the model's strongest signal (Section 6).

2. Data & approach

How the model was built so that its results can be believed

The data

The extract contains **64,738 subscribers** (prepaid and postpaid) with 43 columns covering tenure, plan, spend, recharge behaviour, network experience, complaints, competitor-call share and number-porting enquiries. There are no duplicate subscribers and no unexplained missing values (the text "None" in *home_product* is a genuine category, not missing data, and was preserved). The target is **churn_flag_90d**: whether the subscriber churned within the following 90 days.

Class imbalance: the central challenge

Group	Subscribers	Share
Retained (did not churn)	61,497	94.99%
Churned within 90 days	3,241	5.01%

Only about 1 customer in 20 churns. A model that simply predicts *"nobody will churn"* would be **95.0% accurate** and yet useless - it finds zero churners. For that reason accuracy alone is never used to judge this model; precision, recall, F1 and the area-under-curve scores are the real measures, and we report them all.

How the imbalance was handled

- **Stratified split.** Training, validation and test sets each keep the same 5.01% churn rate, so no set is easier or harder than another.
- **Imbalance-aware tuning.** The model was tuned and stopped on PR-AUC, the score that reflects performance on the rare class (accuracy would hide failure).
- **Class weighting tested, not assumed.** Three weightings (none, moderate, full 19:1) were compared on the validation set; results were within about 0.01 PR-AUC, so the simplest option (no reweighting) was kept. This also keeps predicted probabilities honest.
- **Decision threshold set on validation data.** Rather than the default 50% cut-off, the flagging threshold (0.386) was chosen to maximise F1 on the validation set only. Results at the default 50% are also reported for transparency.
- **No artificial oversampling.** No rows were duplicated or synthesised, so every test customer is a real customer.

Data split (70 / 15 / 15, stratified)

Set	Subscribers	Share	Churned	Churn rate	Purpose
Train	45,316	70%	2,269	5.01%	Model learns patterns
Validation	9,711	15%	486	5.00%	Tuning, early stopping, threshold
Test	9,711	15%	486	5.00%	Final scoring, used once

Leakage control: keeping the result honest

Three columns describe the outcome itself and would make any model look artificially perfect: **churn_reason**, **churn_date** and **churn_flag_30d**. All three were removed before modelling, and the code asserts they are absent. Also removed were *subscriber_id* (an identifier), *circle_code* (duplicate of *circle*) and *join_date* (already captured by tenure). That leaves 36 raw features, expanded to 73 model inputs after encoding categories. As an extra safeguard, no single feature predicts churn near-perfectly (best single-feature AUC is 0.81).

3. Model performance on unseen customers

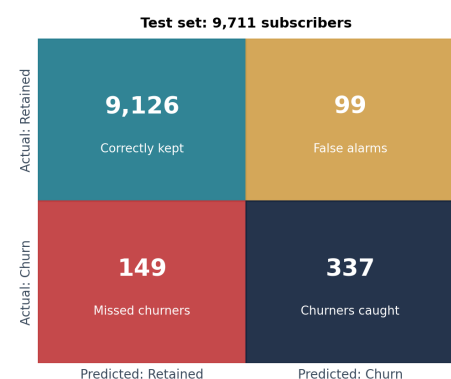
Every metric requested, with a plain-English reading and its 95% confidence interval

Metric	Test score	95% confidence interval	What it means in plain English
Accuracy	0.9745	0.971 - 0.977	Share of all customers classified correctly. High partly because most customers do not churn - read with care.
Precision	0.7729	0.733 - 0.810	Of customers the model flags as churners, this share really do churn. (77 in 100.)
Recall	0.6934	0.652 - 0.733	Of customers who really churn, this share the model catches. (69 in 100.)
F1-score	0.7310	0.697 - 0.760	Single balance score of precision and recall (1.0 = perfect).
ROC-AUC	0.9758	0.971 - 0.981	Pick one churner and one loyal customer at random: the model scores the churner higher this often.
PR-AUC	0.8073	0.778 - 0.837	Quality of ranking on the rare churn class. A random model scores 0.050; ours scores 16 times higher.

Scored at the validation-tuned threshold of 0.386. Confidence intervals come from 1,000 bootstrap resamples of the test set and show the range in which the true performance is expected to lie.

Additional views

Balanced accuracy **0.841** (average of churner-catch rate and loyal-customer-correct rate, immune to class imbalance); Matthews correlation **0.719** (a strict single-number quality score, where 1.0 is perfect); specificity **0.989** (loyal customers correctly left alone). At the default 50% threshold the model is more conservative: precision 0.824, recall 0.638, F1 0.719.



Reading the result

Of 486 customers who really churned, the model **caught 337** and missed 149.

Of 9,225 loyal customers, **9,126** were correctly left alone and only **99** (1.1%) were flagged by mistake.

Each missed churner is a lost customer; each false alarm is an unnecessary offer. Section 5 shows how to dial this trade-off up or down to match the cost of a retention offer.

Figure 1 - Confusion matrix on the test set

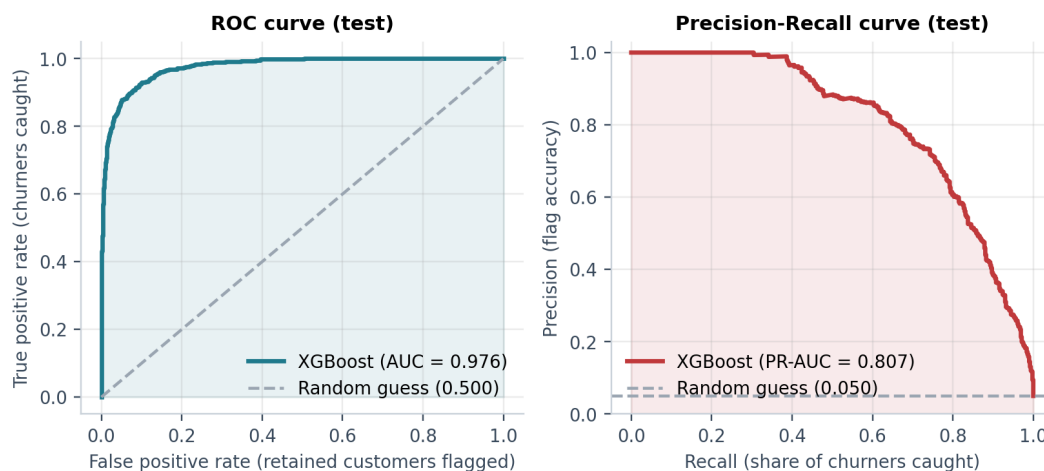


Figure 2 - ROC and Precision-Recall curves on the test set. Curves far above the dashed random-guess lines show strong, real predictive power.

4. Is the model overfitting or underfitting?

The most important reliability question - answered with nine independent checks

Verdict: a good fit

The model is **not overfitted** (it does not memorise the training data) and **not underfitted** (it does not miss real patterns). Performance on unseen customers is almost identical to performance on the customers it learned from, so the results shown to you are what you should expect in use. There is a small, normal training-versus-test difference of about 4 points on PR-AUC and F1 (6 points on precision), which is well inside acceptable limits.

What the two terms mean

Overfitting is like a student who memorises last year's exam: brilliant on questions already seen, poor on new ones. It shows as a large gap between training and test scores. **Underfitting** is a student who did not study enough: weak everywhere, and getting better with more effort. It shows as low scores on both training and test data.

Check 1: training vs validation vs test

Metric	Train	Validation	Test	Gap (train - test)	Reading
ROC-AUC	0.977	0.970	0.976	+0.001	Essentially identical
PR-AUC	0.847	0.809	0.807	+0.040	Small, expected gap
Precision	0.834	0.801	0.773	+0.061	Small, expected gap
Recall	0.719	0.704	0.693	+0.025	Very small gap
F1-score	0.772	0.749	0.731	+0.041	Small, expected gap
Accuracy	0.979	0.976	0.974	+0.004	Essentially identical

A gap under about 0.05 on ROC-AUC is generally considered healthy. Validation and test agree closely, confirming that tuning did not "leak" into the final score.



Figure 3 - Train, validation and test scores side by side. The three bars in each group are nearly level, the signature of a well-fitted model.

Check 2: the learning curve

The learning curve tracks the model as it adds trees. Training error keeps drifting down while validation error flattens and then stays flat. Automatic early stopping halted training at tree 247 (of a permitted 3,000) before validation quality could deteriorate. The validation curve does not turn back up, which is the classic warning sign of overfitting - and it is absent here.

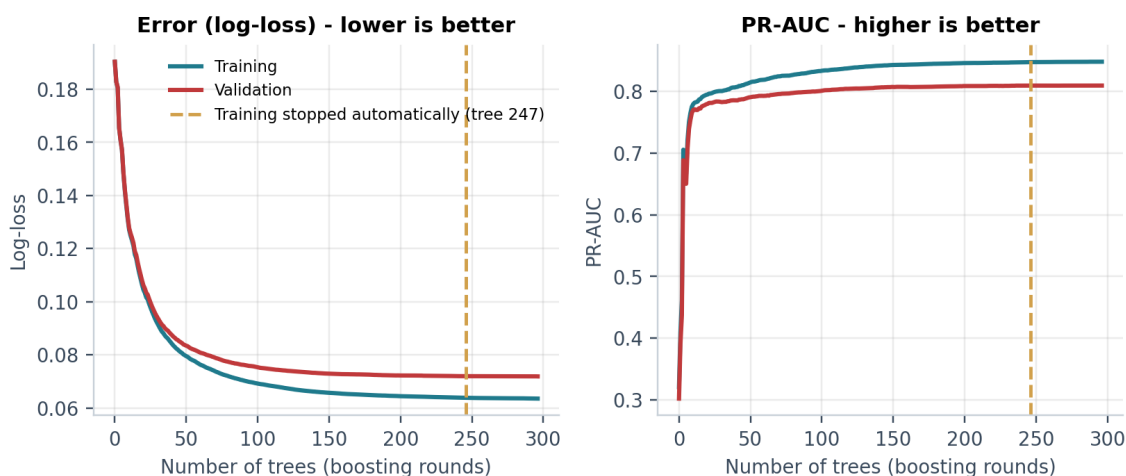


Figure 4 - Training and validation curves stay close and parallel; the dashed line marks the automatic stopping point.

Checks 3 and 4: model complexity and amount of data

We re-trained the model at ten complexity levels (tree depth 1 to 10) and at seven training-set sizes. If the model were **underfitting**, accuracy would rise as complexity increased; if **overfitting**, validation accuracy would fall. Instead validation PR-AUC stays within **0.808 - 0.814** across every depth, and grows only from 0.807 to 0.809 when the last 20% of training data is added. In practice the model has reached the ceiling of what these data can tell us; further complexity or a modest amount of extra data will add little.

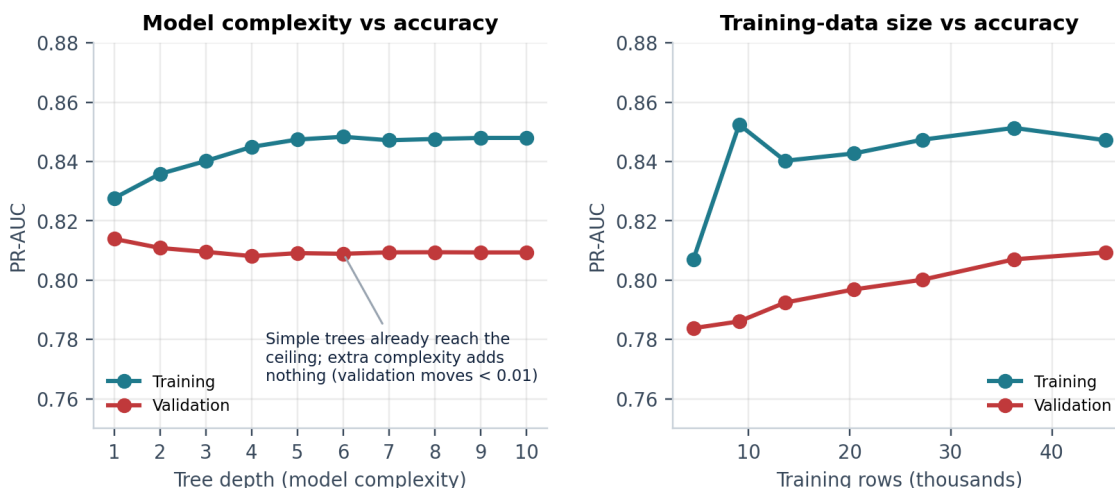


Figure 5 - Left: accuracy is flat regardless of model complexity. Right: accuracy gains flatten as data grows; more data would help only marginally.

The full checklist

#	Check	Evidence	Outcome
1	Train vs test gap	ROC-AUC gap 0.001; PR-AUC gap 0.040	No overfitting
2	Validation vs test agreement	ROC-AUC 0.970 vs 0.976; PR-AUC 0.809 vs 0.807	Consistent
3	Learning curve	Validation flat after tree ~150; early stop at 247	No overfitting
4	Complexity sweep (depth 1-10)	Validation PR-AUC range only 0.006	Robust
5	Training-size sweep	Validation PR-AUC 0.784 to 0.809, flattening	Enough data
6	Tuning stability (30 trials)	Validation ROC-AUC std. dev. 0.0009	Not fragile
7	Underfitting check	Train PR-AUC 0.847 vs 0.050 random; simple logistic regression reaches 0.808 on test	No underfitting
8	Shuffled-label test	ROC-AUC 0.516 when outcomes are scrambled	No leakage
9	Bootstrap confidence	ROC-AUC 95% CI width 0.011	Precise estimate

Why the small gap exists, and why it is controlled

Tree-based models always fit the data they trained on slightly better than new data, so a few points of difference on precision-style metrics is normal. Here it is held in check by deliberate regularisation: leaves need at least **20** customers' worth of evidence, splits must reduce training error by at least **5** units, each tree sees only ~69% of customers and ~53% of features, L1/L2 penalties are applied, and learning is slow (0.05). The result is a model that generalises.

Honest note for the client conversation

A simple logistic-regression benchmark achieves nearly identical results (test ROC-AUC 0.977 vs 0.976; F1 0.723 vs 0.731). This tells us the churn signal in the data is strong and fairly straightforward, not that the model is weak. It is good news: it means the result is stable and easy to explain, and it gives a simple fallback model if ever needed.

5. Business impact

How the model turns into a targeted, cost-efficient retention programme

Focus: where the churners are

Ranking the 9,711 test customers by risk and cutting them into ten equal groups shows how concentrated churn is. The riskiest group (decile 1) churned at **44.0%** against an average of 5.0%, and by decile 3 **98.8%** of all churners have been reached. Deciles 7 to 10 contained no churners at all.

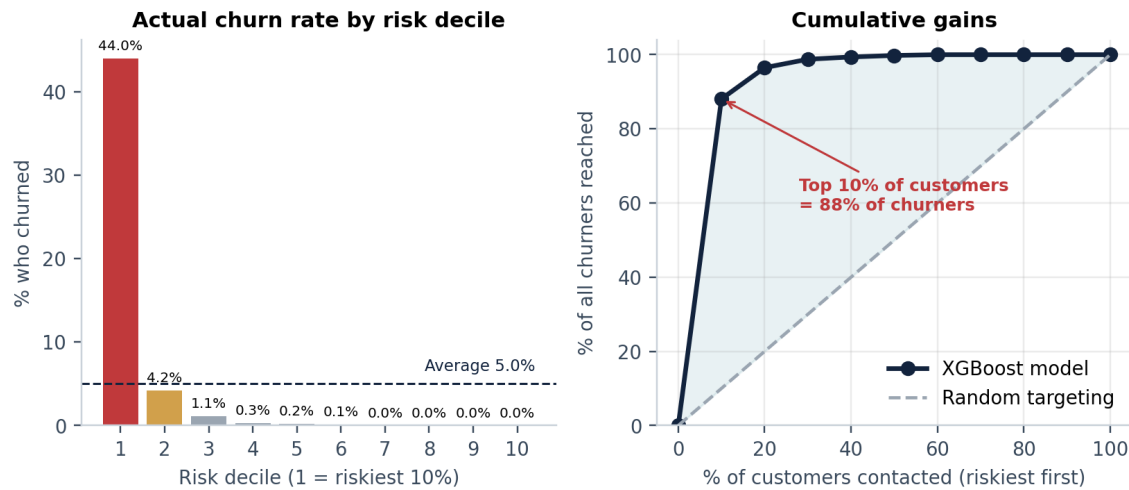


Figure 6 - Left: actual churn rate in each risk decile. Right: cumulative share of churners reached as more customers are contacted.

Risk tiers ready for operational use

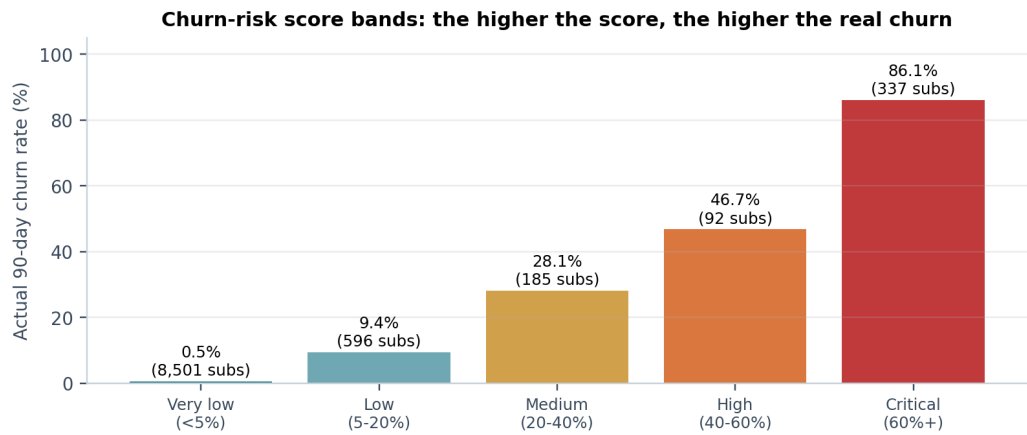


Figure 7 - Real churn rate rises steadily with the model's score, so the score can be used to define retention tiers.

Risk tier	Score band	Customers	Actual churn	Share of all churners	Suggested action
Very low	<5%	8,501	0.5%	9.3%	No spend. Monitor monthly.
Low	5-20%	596	9.4%	11.5%	Light-touch digital engagement.
Medium	20-40%	185	28.1%	10.7%	Automated nudge / small offer.
High	40-60%	92	46.7%	8.8%	Targeted retention offer.
Critical	60%+	337	86.1%	59.7%	Same-day personal outreach.

Test-set counts. The Critical tier alone is 3.5% of customers yet holds about 60% of all churners.

Sizing the campaign: how many customers to contact

Contact volume is a business decision. The table shows what the model delivers at each size, on the test set of 9,711 customers. Contacting fewer customers gives a more accurate list but leaves more churners unreachable; contacting more reaches almost everyone but at lower accuracy.

Contact the riskiest...	Customers contacted	Precision (list accuracy)	Recall (churners reached)	Annual ARPU of churners reached
3% of base	291	87.3%	52.3%	Rs. 4.45 lakh
5% of base	486	73.5%	73.5%	Rs. 6.29 lakh
7% of base	680	58.4%	81.7%	Rs. 6.98 lakh
10% of base	971	44.1%	88.1%	Rs. 7.66 lakh
15% of base	1,457	31.0%	93.0%	Rs. 8.08 lakh
20% of base	1,942	24.2%	96.5%	Rs. 8.45 lakh

Highlighted row: the 10% contact list. The model's own recommended threshold flags 436 customers (4.5%) with 77.3% precision and 69.3% recall. ARPU = last-month ARPU x 12 for the churners in each list.

Revenue at stake (indicative)

Churners in this base are lower-value customers: their average annualised ARPU is **Rs. 1,803** versus **Rs. 2,554** for the average subscriber. Across the full 64,738-subscriber base, churners represent about **Rs. 59.6 lakh** of annualised ARPU. The model flags churners carrying **67%** of that value at its recommended threshold, roughly **Rs. 39.7 lakh** when scaled to the whole base. What is actually saved depends on how many flagged customers accept a retention offer:

Assumed save rate (illustrative)	Revenue retained, full base (Rs. lakh / year)	Break-even cost per flagged customer
10% of flagged churners accept and stay	4.0	Rs. 139
20% of flagged churners accept and stay	7.9	Rs. 279
30% of flagged churners accept and stay	11.9	Rs. 418

Save rates are scenarios, not measured results. Revenue is gross, before campaign cost. Break-even cost per flagged customer = precision x save rate x average annual ARPU of a churner; any retention offer costing less than this pays for itself. Full-base figures are extrapolated from the 15% test sample.

What this means for offer design

Because churners average only about Rs. 150 a month, blanket discounts will erode margin. The model's value is that offers can be reserved for the small group of customers who are both very likely to leave and worth keeping, with low-cost digital nudges for the middle tiers.

6. Churn drivers and key dependency

The model's reasoning in business terms

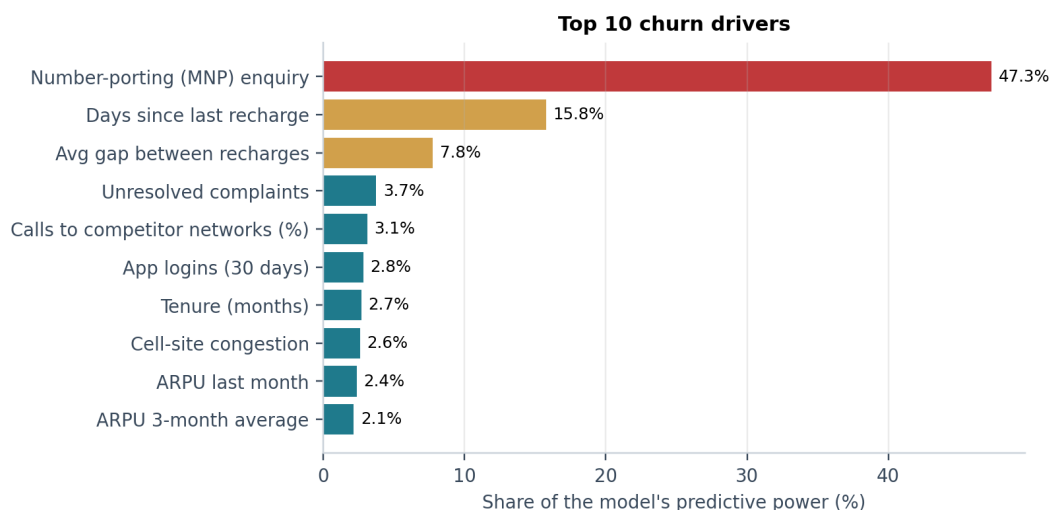
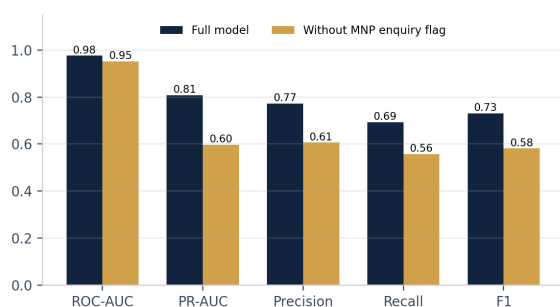


Figure 8 - Share of the model's predictive power attributable to each signal (gain importance). A permutation test on validation data agrees that MNP enquiry and days since last recharge are the two strongest drivers.

Reading the drivers

- **Intent to leave (MNP enquiry).** A customer who has enquired about porting their number is the most powerful warning sign, accounting for almost half of the model's predictive power.
- **Recharge behaviour.** Long gaps since the last recharge, and long average gaps between recharges, together contribute about a quarter of the signal: customers drift away before they formally leave.
- **Service friction and competitor exposure.** Unresolved complaints, a high share of calls to competitor networks, and congested cell sites each add real but smaller signal.
- **Spend level matters little on its own.** ARPU appears but is lower down the list; churn is driven by behaviour and intent, not by how much a customer spends.

Dependency test: what if the MNP signal were not available?



We re-trained the identical model with the MNP enquiry flag removed. ROC-AUC stays strong at **0.951** (from 0.976), showing that the model still ranks risk well from behaviour alone. Precision-oriented scores fall more: PR-AUC 0.60 (from 0.81), F1 0.58 (from 0.73).

Why this matters: if the MNP enquiry is logged only a few days before the port-out, the model behaves more like a confirmation of imminent churn than an early warning. If it is logged weeks earlier, it is a genuine early-warning signal. We recommend confirming the recording timing with the network/porting team before go-live.

Figure 9 - Test performance with and without the MNP enquiry flag.

Recommended safeguard

Run the model in two views: the full model for same-day save desk outreach, and the behaviour-only model as the earlier, broader nudge list. Together they cover both imminent leavers and those who are drifting.

7. Risks, recommendations and next steps

Transparent limitations, and a practical path to value

Limitations we want you to know about

Limitation	What it means and how we manage it
Random split, single snapshot	Data were split at random, not by date. Live performance can be lower if customer behaviour shifts. Mitigation: validate on next month's data (out-of-time test) before scaling.
Prediction is not persuasion	The model finds who is likely to leave, not who can be persuaded to stay. Mitigation: run the campaign with a randomly held-out control group to measure the true save rate.
Threshold reflects a chosen balance	The 0.39 threshold maximises F1. If offers are cheap, lower it to reach more churners; if expensive, raise it. See the contact-volume table.
MNP flag timing	Strongest signal; its recording time relative to port-out must be confirmed (Section 6).
Revenue figures are indicative	Save rates and full-base scaling are assumptions to be replaced by measured values after the pilot.
Linear benchmark ties on accuracy	A simple logistic regression performs almost as well. XGBoost is delivered as requested and is ready for a controlled pilot, with the simpler model as a fallback.

Recommended rollout

#	Action	When	Detail
1	Confirm MNP timing	Weeks 0-1	Validate when the flag is logged; decide whether to run the two-view setup.
2	Pilot with a control group	Weeks 1-8	Contact Critical and High tiers; hold out a random 10-20% as control to measure the true save rate and cost.
3	Choose the contact volume	Week 2	Use the contact-volume table with the actual offer cost to pick the threshold.
4	Score monthly	Ongoing	Re-score the base every month; feed the risk list to the retention desk and CRM.
5	Monitor and retrain	Quarterly	Track precision in the top decile, score drift and churn rate; retrain when performance falls.
6	Validate out-of-time	Month 2	Score a later month and compare to these results to confirm stability over time.

Conclusion

The model separates likely churners from loyal customers with a ROC-AUC of 0.976, finds about 69% of churners at 77% precision, and reaches 88% of churners by contacting just 10% of customers. It is well fitted, leakage-free and stable. With the MNP timing confirmed and a controlled pilot, it is ready to guide a targeted retention programme.

Appendix: method, settings and glossary

For technical reviewers and reproducibility

Method summary

- **Algorithm:** XGBoost gradient-boosted trees (binary logistic objective, histogram tree method).
- **Target:** churn_flag_90d. **Excluded:** churn_reason, churn_date, churn_flag_30d (outcome-linked); subscriber_id, circle_code, join_date (identifier / redundant).
- **Encoding:** categorical fields (circle, zone, device brand, home product, plan type) one-hot encoded using levels from the training set only; 73 model inputs.
- **Split:** stratified 70 / 15 / 15 with a fixed random seed (42), no subscriber appears in more than one set.
- **Tuning:** 30 random configurations, each trained with early stopping (50 rounds) on validation PR-AUC; best configuration chosen by validation PR-AUC.
- **Threshold:** 0.3863, the F1-maximising value on the validation set. The test set was scored once and never used for any choice; confidence intervals use 1,000 bootstrap resamples.
- **Sanity checks:** shuffled-label test, single-feature leakage scan, majority-class and logistic-regression baselines, ablation without the MNP flag.

Selected hyper-parameters

Parameter	Value	Parameter	Value
Learning rate	0.05	Row subsample	0.694
Max tree depth	7	Column subsample	0.529
Trees used (early stopping)	247	L2 penalty (lambda)	3
Min child weight	20	L1 penalty (alpha)	2
Gamma (min split gain)	5	Class weight (scale_pos_weight)	1

Comparison with baselines (test set)

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Predict nobody churns	0.950	0.000	0.000	0.000	0.500	0.050
Logistic regression (balanced)	0.973	0.747	0.700	0.723	0.977	0.808
XGBoost (this work)	0.974	0.773	0.693	0.731	0.976	0.807

The do-nothing model scores 95% accuracy with zero churners found, which is why accuracy alone is not used to judge this model.

Glossary

Precision / Recall	Precision: of customers flagged as churners, the share who really churn. Recall: of customers who really churn, the share flagged.
ROC-AUC	Probability the model ranks a random churner above a random loyal customer. 0.5 = guessing, 1.0 = perfect.
PR-AUC	Ranking quality on the rare churn class; baseline equals the churn rate (0.050).
Lift	How many times more churners a targeted list contains than a random list of the same size.
Overfitting / Underfitting	Memorising training data (poor on new data) / failing to learn real patterns (poor everywhere).
Data leakage	Using information unavailable at prediction time, which produces falsely perfect results.
MNP / ARPU	MNP: Mobile Number Portability (moving a number to another operator). ARPU: average revenue per user, annualised here as last-month ARPU x 12.

Comparability: the earlier exploratory report quoted an indicative AUC of 0.937 and 73% top-decile capture without documenting its model settings, so those figures are not directly comparable. All figures here come from the reproducible script `churn_xgboost_model.py`.